C++ primer for a Python developer

Every C++ feature used in StrataKV's code, explained next to the Python you already know. The biggest mental shift: in C++ you decide where every object lives and when it dies, and the compiler checks types before the program ever runs.

How C++ code becomes a program

- **Header files (`.h`)** declare what exists: class shapes and function signatures. **Source files (`.cc`)** define how they work. Other files `#include` the header to use them. Python has no split; a module is both.
- **Compiling** turns each `.cc` file into machine code; **linking** joins them into one program or library. Python skips both and interprets at runtime.
- **CMake** is the build tool that runs the compiler and linker with the right settings, like a much more powerful `pyproject.toml` build section. **Presets** are named configurations: `debug`, `release`, `asan`, `tsan`.
- **Undefined behavior (UB):** some mistakes, like reading freed memory or overflowing a signed integer, aren't errors in C++; the program just does something unpredictable. **Sanitizers** are compiler modes that catch these at runtime: ASan (memory errors), UBSan (undefined behavior), TSan (data races). That's why every phase must pass under them.

Syntax used in the learning guide

C++	What it means	Python closest	Seen in snippet
`uint8_t`, `uint32_t`, `uint64_t`	Unsigned integers of exactly 8, 32, 64 bits	`int` (unbounded) plus `struct.pack`	1, 4
`auto x = f();`	Let the compiler infer the type	`x = f()`	3, 6
`const T&`	Read-only reference: use the caller's object without copying it	Passing any object (no copy)	1, 6

C++	What it means	Python closest	Seen in snippet
`T*` and `&x`	Pointer to a `T`; `&x` is the address of `x`; `p->field` reads through a pointer	Object references	3, 6
`std::string`	Owns its characters	`str` / `bytes`	3, 6
`std::string_view`	A read-only window into someone else's characters; no copy, no ownership	`memoryview`	4, 6
`std::span<const uint8_t>`	A window into a contiguous array of bytes	`memoryview` over `bytes`	1
`std::vector<T>`	Growable array that owns its items	`list`	1, 6
`if (auto s = f(); !s.ok()) return s;`	Declare `s`, then test it, in one line; `s` exists only inside the `if`	`if not (s := f()).ok(): return s`	1, 2
`auto [group, last] = f();`	Structured binding: unpack a returned pair	`group, last = f()`	8
`[&](Iterator* a, Iterator* b) { ... }`	Lambda (anonymous function); `[&]` captures surrounding variables by reference	`lambda a, b: ...`	6, 8
`std::function<void(X)>`	Holds any callable with that signature	A `Callable[[X], None]`	1

C++	What it means	Python closest	Seen in snippet
`std::move(x)`	Hand `x`'s contents to someone else instead of copying; `x` is left empty	No equivalent; Python never copies on assignment	3
`namespace py = pybind11;`	Short alias for a namespace	`import pybind11 as py`	9
`static_cast<uint32_t>(n)`	Explicit, checked type conversion	`int(n)`	1

Memory and ownership (the part with no Python equivalent)

Python's garbage collector frees objects when nothing references them. C++ has no garbage collector; instead, objects are tied to **scopes** and **owners**.

```
void Example() {
  std::string a = "hello";                        // lives on the stack; freed
at the closing }
  auto b = std::make_unique<MemTable>();          // one owner; freed when b
goes away
  auto c = std::make_shared<const Version>();     // shared owners; freed when
the LAST one goes away
  auto d = c;                                     // second owner of the same
Version (refcount 2)
}                                                 // a, b, c, d all cleaned up
here, in reverse order
```

- **RAII** ("Resource Acquisition Is Initialization"): an object acquires a resource in its constructor and releases it in its destructor, which runs automatically when the object goes out of scope. File handles, locks, and memory all work this way. Python's `with` statement is the closest idea, except RAII needs no special syntax.
- **`std::unique_ptr<T>`**: exactly one owner. Can be moved, not copied. Use by default.
- **`std::shared_ptr<T>`**: reference-counted, like every Python object. StrataKV uses it for `Version` snapshots (learning guide snippet 7) so files stay alive while any reader still needs them.

- **Raw pointers (`T*`)** don't own anything; they just point. The skip list uses raw pointers into an **arena**, a big block of memory freed all at once when the memtable is discarded.
- **The classic bugs these rules prevent:** use-after-free (reading memory already released), double free, and leaks. ASan catches all three.

Concurrency primitives

C++	What it does	Python closest
`std::thread`	A real OS thread; runs truly in parallel (no GIL in C++)	`threading.Thread`, minus the GIL
`std::mutex` + `std::lock_guard lock(mu);`	Exclusive lock, released automatically at the end of the scope (RAII)	`with lock:`
`std::unique_lock`	Like `lock_guard`, but can unlock and relock mid-scope; required by condition variables	`lock.acquire()` / `release()`
`std::shared_mutex`	Many readers or one writer	No stdlib equivalent
`std::condition_variable cv; cv.wait(lock, pred);`	Sleep until another thread signals and `pred` is true	`threading.Condition.wait_for(pred)`
`std::atomic<T>`	A value that threads can read and write without a lock, safely	No equivalent; the GIL hides the need

C++	What it does	Python closest
`memory_order_acquire` / `release`	Ordering rules for atomics: a release store "publishes" everything written before it to any thread that does an acquire load of that value	No equivalent
`memory_order_relaxed`	Atomic but with no ordering promises; only safe when nothing else depends on it	No equivalent

Why memory ordering exists. CPUs and compilers reorder memory operations for speed. On one thread you never notice. Across threads, a reader could see a pointer to a new skip-list node before seeing the node's contents. Release on the writer plus acquire on the reader forbids that reordering. ARM chips, like Apple Silicon, reorder more aggressively than x86, so a missing ordering bug can hide on one machine and appear on another.

Encoding bytes on disk

- **Fixed-width little-endian:** a `uint32_t` is written as 4 bytes, least significant first. Python: `struct.pack("<I", n)`.
- **Varint:** small numbers in fewer bytes. Each byte carries 7 bits of the number, and its top bit says "more bytes follow." 1 takes 1 byte; 300 takes 2. Saves space for key and value lengths, which are usually small.
- **CRC32C:** a checksum, a 4-byte fingerprint of data. If any bit changes, the checksum almost certainly changes. It detects corruption and torn writes; it is not a cryptographic hash.

pybind11 and the Python side

- **pybind11** is a C++ library that exposes C++ classes and functions to Python. `PYBIND11_MODULE(_stratakv, m)` defines a module importable as `_stratakv`; `py::class_<DB>(m, "DB").def("get", ...)` adds a class with a method.
- **Type conversion:** pybind11 converts between `py::bytes` and `std::string`, Python `int` and C++ integers, `None` and empty values. Each conversion copies, which matters for very large values.
- **The GIL:** CPython's lock that lets only one thread run Python code at a time. C++ code can release it (`py::gil_scoped_release`) during long work so other Python threads keep running, but must not touch Python objects while released.
- **Exceptions across the boundary:** C++ `std::runtime_error` becomes Python `RuntimeError`; `py::register_exception` creates custom Python exception types like `stratakv.CorruptionError`.
- **scikit-build-core:** the build backend that makes `pip install .` run CMake and place the compiled `.so` file inside the Python package.
- **The thin Python wrapper** (`python/stratakv/__init__.py`) adds Pythonic touches on top: type hints, a context manager (`with stratakv.DB.open(path) as db:`), and the `MutableMapping` interface so `db[k] = v` works like a dict.